Spring Boot - Interview Questions

Spring & Spring Boot Core

1] What is the Spring Framework and why is it used in Java applications?

2] What problem does Spring Framework solve?

3] What are the core modules of the Spring Framework?

4] What is Spring Boot and how is it different from Spring Framework?

5] Why was Spring Boot introduced?

6] What problems does Spring Boot solve?

7] What are the key differences between Spring and Spring Boot?

8] Why is Spring Boot preferred over traditional Spring development?

9] How does configuration differ in Spring vs Spring Boot?

10] What are the main features of Spring Boot?

11] What is meant by “convention over configuration” in Spring Boot?

12] What is the role of starters and auto-configuration in Spring Boot?

13] What are the advantages (pros) of using Spring Boot?

14] What are some disadvantages (cons) of Spring Boot?

15] In which scenarios would you avoid using Spring Boot?

16] What are the different ways to create a Spring Boot application?

17] What is Spring Initializr and when do you use it?

18] How can you manually create a Spring Boot project?

19] What is a Spring Boot Starter?

20] Why do we need starter dependencies?

21] What is the purpose of spring-boot-starter-web, spring-boot-starter-data-jpa, etc.?

22] What is the Start Class in a Spring Boot application?

23] Why must the Start Class be placed in the root package?

24] How does the Start Class trigger component scanning?

25] What does @SpringBootApplication contain internally?

26] What is the role of @EnableAutoConfiguration?

27] What does @SpringBootConfiguration do?

28] What is the use of @ComponentScan inside @SpringBootApplication?

29] What happens internally when SpringApplication.run(…) is executed?

30] How does the run() method create and initialize the application context?

31] What is the role of SpringApplication class?

32] What is meant by Spring Boot application bootstrapping?

33] What are the steps followed during bootstrapping process?

34] What is the role of the main() method in bootstrapping?

35] What is AutoConfiguration in Spring Boot?

36] How does Spring Boot decide which auto-configurations to apply?

37] What is spring.factories or AutoConfiguration imports?

38] How do you disable a specific autoconfiguration?

39] What is IoC (Inversion of Control)?

40] How does the IoC container work in Spring Boot?

41] What is the difference between BeanFactory and ApplicationContext?

42] What is Dependency Injection?

43] What are the different types of dependency injection in Spring Boot?

44] Why is constructor injection preferred over field injection?

45] What are stereotype annotations in Spring?

46] What is the difference between @Component, @Service, and @Repository?

47] Why do we use stereotype annotations?

48] Why should the base package follow a naming convention?

49] Why should the main class be placed in the base/root package?

50] How does incorrect package structure break component scanning?

51] What is component scanning in Spring Boot?

52] How does @ComponentScan work?

53] How do you include or exclude specific packages in component scanning?

54] What is autowiring in Spring Boot?

55] How does Spring resolve dependencies automatically?

56] What happens if multiple beans of the same type exist?

57] What is the purpose of @Qualifier annotation?

58] When should you use @Qualifier?

59] Can @Qualifier and @Primary work together?

60] What does @Primary do?

61] When should you use @Primary instead of @Qualifier?

62] What happens when @Primary is applied to multiple beans?

63] What is the @Configuration annotation used for?

64] How is @Configuration different from @Component?

65] What is the purpose of proxyBeanMethods attribute inside @Configuration?

66] What is @Bean used for?

67] How is a bean created using @Bean annotation?

68] How do @Bean and @Configuration work together?

69] Can @Bean methods call other @Bean methods?

70] What is a Spring Bean lifecycle?

71] What are init() and destroy() callbacks?

72] What are BeanPostProcessor and BeanFactoryPostProcessor?

73] What is the lifecycle of singleton vs prototype beans?

74] What are different bean scopes available in Spring Boot?

75] What is the default bean scope?

76] What is the difference between singleton and prototype scopes?

77] What is the Spring Boot banner?

78] How do you customize the Spring Boot banner?

79] How do you disable the banner?

80] What is a standalone Spring Boot application?

81] How does Spring Boot enable standalone development?

82] What components are mandatory for a standalone app?

83] What is layered architecture in a Spring Boot application?

84] What are the typical layers (Controller, Service, Repository)?

85] Why is layered architecture recommended?

86] What is CommandLineRunner?

87] What is ApplicationRunner?

88] What is the difference between CommandLineRunner and ApplicationRunner?

89] When should you use runners in Spring Boot?

90] How do you execute code at application startup using Runner?

Spring Boot Data JPA

91] What is Spring Data JPA and why do we use it?

92] What is the difference between JPA, Hibernate, and Spring Data JPA?

93] What is the role of EntityManager in JPA? Does Spring Data JPA use it internally?

94] What is a Persistence Context?

95] What is an Entity in JPA? What are the rules to define an entity?

96] What is CrudRepository, JpaRepository, and PagingAndSortingRepository? Differences?

97] Which repository would you choose for pagination? Why?

98] How does Spring Data create implementations of Repository interfaces automatically?

99] What are derived query methods? Explain findByXXX naming rules.

100] What is the difference between getOne(), getById() and findById()?

101] How do you write JPQL queries using @Query annotation?

102] What is the difference between JPQL and Native SQL queries?

103] What is @Modifying annotation? When is it required?

104] How do you execute INSERT/UPDATE/DELETE operations in Spring Data JPA?

105] How to call stored procedures in Spring Data JPA?

106] What is @Transactional and why do we need it?

107] What happens if a runtime exception occurs inside a @Transactional method?

108] What is transaction propagation? Explain REQUIRED, REQUIRES_NEW.

109] What is transaction isolation? Explain READ_COMMITTED, SERIALIZABLE.

110] What is the purpose of @Table and @Column annotations?

111] Explain @Id, @GeneratedValue, and different GenerationTypes.

112] What is the difference between SEQUENCE and TABLE strategies?

113] Explain @Embeddable and @Embedded.

114] What is @Enumerated and the difference between ORDINAL and STRING?

115] Explain @OneToOne, @OneToMany, @ManyToOne, and @ManyToMany with examples.

116] What is cascading? Explain CascadeType.ALL, PERSIST, MERGE, REMOVE.

117] What is FetchType LAZY vs EAGER? Which is default for each mapping?

118] What is the purpose of mappedBy?

119] What is orphanRemoval?

120] How do you implement pagination in Spring Data JPA?

121] What is Page, Pageable, and PageRequest?

122] Difference between Slice and Page?

123] What is JPA Criteria API? When should we use it?

124] What is Specification in Spring Data JPA?

125] How do you create dynamic queries in Spring Data JPA?

126] What is N+1 select problem? How to avoid it?

127] What is the difference between join fetch and normal join?

128] What is 1st level cache and 2nd level cache in JPA?

129] How do you enable Hibernate second-level cache in Spring Boot?

130] Why should LAZY fetching be preferred over EAGER fetching?

131] What is the purpose of spring.jpa.hibernate.ddl-auto?

132] What is the use of spring.jpa.show-sql?

133] What is the purpose of spring.jpa.database-platform (dialect)?

134] How does Spring Boot auto-configure Spring Data JPA?

135] How to configure multiple data sources in Spring Boot?

136] What is optimistic and pessimistic locking?

137] Explain @Version and how optimistic locking works.

138] What is dirty checking in Hibernate?

139] Explain the lifecycle of an Entity (Transient, Managed, Detached, Removed).

140] What is flush and clear in EntityManager?

141] What is an embedded database and why is it useful for development/testing?

142] How do you develop a Spring Boot application using an embedded database like H2?

143] How do you develop a Spring Boot application using a MySQL database?

144] What are profiles in Spring Boot and how are they used to manage environments?

Spring Profiles

145] What is a Profile in Spring? Why do we use it?

146] How do you define a profile in Spring Boot?

147] What is the purpose of the @Profile annotation?

148] How do you activate a specific profile at runtime?

149] What is the difference between spring.profiles.active and spring.profiles.default?

150] How do you define profile-specific properties files in Spring Boot?

151] What is the naming convention for profile-specific property files?

152] How does Spring decide which bean to load when multiple beans have different profiles?

153] Can a bean belong to multiple profiles? How?

154] What is the use of !profileName inside @Profile?

155] How do profiles work with YAML files?

156] What happens if no profile is active in Spring Boot?

157] How do you activate multiple profiles at the same time?

158] Can we activate profiles through program arguments? How?

159] How do you use @ActiveProfiles in test cases?

160] What is the difference between @Profile and @ConditionalOnProperty?

161] Can we use profiles to switch between different databases? Explain.

162] What happens when two profile-specific property files contain the same key?

163] Can profile conditions be combined (AND / OR)? How?

164] How do you create custom profile-based configuration classes?

Spring MVC

165] What is Spring Web MVC and what are its main components?

166] How does Spring Web MVC differ from traditional Servlet-based web applications?

167] Explain the request-response flow in Spring Web MVC.

168] What is the role of DispatcherServlet in Spring MVC?

169] What are the key advantages of using Spring Web MVC over other frameworks?

170] How does Spring MVC support loose coupling between components?

171] How does Spring MVC simplify web application development?

172] Why is Spring MVC considered more maintainable than JSP/Servlet-based applications?

173] Explain the architecture of Spring MVC with a diagram.

174] What are the major layers/components in Spring MVC architecture?

175] How does the request flow from client to view in Spring MVC?

176] What is the role of DispatcherServlet, Controller, Model, and View in the architecture?

177] What is the Front Controller pattern in Spring MVC?

178] How does DispatcherServlet act as a Front Controller?

179] What are the responsibilities of the Front Controller?

180] Why is the Front Controller pattern important in web frameworks?

181] What is a Controller in Spring MVC?

182] How do you define a Controller in Spring Boot?

183] Difference between @Controller and @RestController.

184] How does a Controller handle HTTP GET and POST requests?

185] What is the role of @RequestMapping annotation in Controllers?

186] What is a Handler Mapping in Spring MVC?

187] How does Spring MVC determine which controller handles a request?

188] Explain the role of RequestMappingHandlerMapping in Spring MVC.

189] What are the types of handler mappings available in Spring MVC?

190] What is a View Resolver in Spring MVC?

191] How does Spring MVC resolve view names to actual views?

192] Explain the difference between InternalResourceViewResolver and ThymeleafViewResolver.

193] How can you configure multiple view resolvers in Spring Boot?

194] How do you create a Spring Boot web application?

195] What are the main components required in a Spring Boot web application?

196] How does Spring Boot simplify web application development compared to Spring MVC?

197] How do embedded servers help in Spring Boot web applications?

198] What is an embedded HTTP server in Spring Boot?

199] What are the advantages of using embedded servers like Tomcat, Jetty, or Undertow?

200] How does Spring Boot configure an embedded server by default?

201] Can you run a Spring Boot application without an embedded server? How?

202] How do you replace Tomcat with Jetty in a Spring Boot application?

203] Which dependency is required to make Jetty the default server?

204] How does Spring Boot auto-configure Jetty as the server?

205] Are there any limitations when using Jetty instead of Tomcat?

206] How do you deploy a Spring Boot web application on an external server like Tomcat?

207] What packaging type should you use to deploy to an external server?

208] How do you disable the embedded server when deploying externally?

209] Explain the steps to convert a Spring Boot JAR into a WAR for external deployment.

210] How can data be sent from an HTML form to a Spring Boot controller?

211] What is the difference between @RequestParam and @ModelAttribute?

212] How do you bind form data to a Java object in Spring MVC?

213] How is JSON data sent from UI handled in a controller using @RequestBody?

214] How do you send data from a Spring controller to the UI?

215] What is the role of Model and ModelMap in Spring MVC?

216] How do you send JSON response data to the UI?

217] Difference between returning a ModelAndView vs a String view name.

218] What is the purpose of @RequestBody in Spring MVC?

219] How does Spring MVC convert HTTP request body to a Java object?

220] Can @RequestBody handle JSON and XML data? How?

221] Difference between @RequestBody and @ModelAttribute.

222] What is the purpose of @ResponseBody in Spring MVC?

223] How does @ResponseBody work with JSON and XML responses?

224] Difference between @ResponseBody and returning a ModelAndView.

225] How does @RestController simplify the use of @ResponseBody?

226] How do you create a form-based application in Spring Boot?

227] How do you bind form fields to a Java object?

228] How do you handle form submission in a controller?

229] How do you perform validation on form inputs in Spring Boot?

230] What is Thymeleaf and why is it used in Spring Boot?

231] How does Thymeleaf differ from JSP?

232] What are the advantages of using Thymeleaf for Spring Boot applications?

233] What are some common Thymeleaf expressions and syntax?

234] How do you integrate Thymeleaf in a Spring Boot web application?

235] How do you pass data from the controller to a Thymeleaf template?

236] How do you handle form submission with Thymeleaf templates?

237] How do you conditionally display content in Thymeleaf?

238] How can you send emails in Spring Boot applications?

239] What dependencies are required for sending emails?

240] How do you configure SMTP server properties in Spring Boot?

241] How do you send HTML emails using Spring Boot?

242] How do you handle exceptions globally in a Spring Boot web application?

243] What is the purpose of @ControllerAdvice?

244] How do you use @ExceptionHandler to handle specific exceptions?

245] How can you send custom error responses (JSON) to the client?

246] Difference between local exception handling in a controller and global exception handling.

Spring Rest:

Distributed Applications & Web Services

247] What is a distributed application?

248] What are the main characteristics of distributed applications?

249] Give some examples of distributed applications in real-time systems.

250] How do distributed applications differ from monolithic applications?

251] What are the common distributed technologies used in Java?

252] Explain RMI, CORBA, and Web Services briefly.

253] What is the role of message brokers in distributed systems?

254] How do distributed technologies ensure scalability and fault tolerance?

255] What is the difference between SOAP and REST?

256] What are the advantages and disadvantages of SOAP?

257] When would you choose REST over SOAP?

258] How does SOAP use XML and REST support JSON?

259] What is a RESTful service?

260] What are the key components of REST architecture?

261] What are the advantages of REST over other web services?

262] Explain how REST handles CRUD operations.

263] What are the main principles of REST?

264] Explain statelessness in REST.

265] What is uniform interface in REST?

266] What is resource-based architecture in REST?

267] What are one-time operations in web services?

268] Give examples of operations that are typically one-time.

269] How does idempotency relate to one-time operations?

270] What are run-time operations in distributed services?

271] How do run-time operations differ from one-time operations?

272] How are these operations managed in REST APIs?

XML, JSON, and Data Binding

273] What is JAX-B and why is it used?

274] How does JAX-B simplify XML processing in Java?

275] What are the main annotations in JAX-B?

276] Explain the architecture of JAX-B.

277] What are the roles of JAXBContext, Marshaller, and Unmarshaller?

278] How does JAX-B handle XML binding with Java classes?

279] How do you generate Java classes from XML using JAX-B?

280] How do you marshal and unmarshal XML using JAX-B?

281] Can JAX-B be used with REST APIs? How?

282] What is JSON and why is it used?

283] How does JSON differ from XML?

284] Explain the structure of JSON objects and arrays.

285] What are the key differences between XML and JSON?

286] Which one is more efficient for web services and why?

287] How does parsing differ between XML and JSON?

288] What is Jackson API in Java?

289] How does Jackson help in converting Java objects to JSON?

290] What are the main classes in Jackson library?

291] How do you handle custom serialization and deserialization in Jackson?

292] How do you convert a Java object to JSON using Jackson?

293] How do you convert JSON to Java object using Jackson?

294] What exceptions should you handle while using Jackson?

295] How do you serialize a list of objects using Jackson?

296] What is GSON and how is it different from Jackson?

297] How do you convert Java objects to JSON using GSON?

298] How do you convert JSON strings to Java objects using GSON?

299] Explain how to serialize Java objects using GSON.

300] How do you deserialize JSON arrays using GSON?

301] How do you handle custom field naming in GSON?

HTTP & REST API Basics

302] What is HTTP and how does it work?

303] Explain the HTTP request-response lifecycle.

304] What are HTTP headers and why are they important?

305] What are the main HTTP methods and their purpose (GET, POST, PUT, DELETE, PATCH)?

306] Which HTTP methods are idempotent and which are not?

307] How do you use HTTP methods in REST API design?

308] What are common HTTP status codes in REST APIs?

309] What is the difference between 4xx and 5xx status codes?

310] How do you handle custom error codes in REST APIs?

Spring Boot REST Controllers & Annotations

311] What is @RestController in Spring Boot?

312] How does @RestController differ from @Controller?

313] When would you use @RestController in web applications?

314] What is @RequestBody used for in Spring Boot?

315] How does Spring convert JSON payload into a Java object using @RequestBody?

316] Can @RequestBody handle XML payloads?

317] What is the purpose of @ResponseBody?

318] How is @ResponseBody related to @RestController?

319] Can @ResponseBody return JSON and XML?

320] What is @RequestParam used for?

321] How do you set default values for @RequestParam?

322] What happens if a required @RequestParam is missing?

323] What is @PathVariable in Spring Boot?

324] How is @PathVariable different from @RequestParam?

325] Can you use multiple @PathVariables in a URL?

326] What is MediaType in Spring Boot?

327] How does MediaType help in content negotiation?

328] What are common MediaTypes used in REST APIs?

329] What does consumes attribute in @RequestMapping do?

330] How do you restrict a REST endpoint to accept only JSON?

331] Can consumes accept multiple media types?

332] What does produces attribute in @RequestMapping do?

333] How do you return JSON or XML from the same endpoint?

334] Can you specify multiple response types in produces?

335] What is the Accept header in HTTP requests?

336] How does Spring Boot use the Accept header to determine response type?

Content-Type, REST Project Setup & Testing

338] What is the purpose of Content-Type header?

339] How is Content-Type used in POST and PUT requests?

340] What is the difference between Content-Type and Accept headers?

341] How do you develop a REST API using Spring Boot?

342] What annotations are used to define REST endpoints?

343] How do you handle CRUD operations in Spring Boot REST API?

344] How do you structure a Spring Boot REST project?

345] What is Postman and how is it used for testing REST APIs?

346] How do you send GET, POST, PUT, DELETE requests using Postman?

347] How do you test APIs with query parameters, headers, and body?

348] What is Swagger and why is it used?

349] How do you integrate Swagger UI with Spring Boot?

350] What is the difference between Swagger and OpenAPI?

351] How does Swagger help with API documentation?

Exception Handling in REST APIs

352] How do you handle exceptions globally in Spring Boot REST APIs?

353] What is the use of @ControllerAdvice?

354] How do you use @ExceptionHandler for custom exceptions?

355] How do you return meaningful JSON error responses to clients?

Reactive Programming & REST Clients

356] What is Mono in reactive programming?

357] How does Mono differ from a normal Java object?

358] When should you use Mono instead of Flux?

359] What is Flux in reactive programming?

360] How is Flux different from Mono?

361] How do you emit multiple values using Flux?

362] How do you handle backpressure in Flux?

363] What is a REST client in Spring Boot?

364] When would you use a REST client in an application?

365] Name some REST clients provided by Spring Boot.

366] What is RestTemplate in Spring Boot?

367] How do you perform GET, POST requests using RestTemplate?

368] Is RestTemplate synchronous or asynchronous?

369] How do you handle headers and query parameters with RestTemplate?

370] What is WebClient in Spring Boot?

371] How does WebClient differ from RestTemplate?

372] How do you perform reactive REST calls using WebClient?

373] How do you handle responses and errors with WebClient?

374] Compare RestTemplate and WebClient.

375] Which one is preferred for reactive applications and why?

376] Can RestTemplate be used in non-blocking calls?

377] What is reactive programming in Spring?

378] What are the main benefits of reactive programming?

379] What is the difference between reactive and traditional synchronous programming?

380] Name some reactive libraries in Java.

381] What is the difference between synchronous and asynchronous REST calls?

382] Give an example of a synchronous call using RestTemplate.

383] Give an example of an asynchronous call using WebClient.

384] What are the advantages of asynchronous calls?

Messaging & Caching

385] What is Apache Kafka and why is it used?

386] How do you integrate Kafka with Spring Boot?

387] What are producers and consumers in Kafka?

388] How does Kafka handle message reliability and ordering?

389] How do you integrate Redis cache with Spring Boot?

390] What is the purpose of caching in REST APIs?

391] How do you use @Cacheable and @CacheEvict in Spring Boot?

392] How does Redis improve application performance?

HATEOAS, Versioning & Testing

393] What is HATEOAS and how is it implemented in Spring Boot REST APIs?

394] How do you handle versioning of REST APIs in Spring Boot?

395] How do you unit test REST controllers using MockMvc?

396] How do you test WebClient reactive endpoints using WebTestClient?

397] What are the different strategies for REST API versioning (URI, request parameter, header)?

398] How do you implement conditional requests using ETag and If-None-Match in Spring Boot?

399] How do you write integration tests for REST APIs in Spring Boot?

400] How do you mock external REST services while testing Spring Boot applications?

Spring Boot Actuator

401] What is Spring Boot Actuator?

402] Why do we use Spring Boot Actuator in applications?

403] What are the benefits of using Actuator?

404] Is Spring Boot Actuator part of Spring Boot core, or do you need to add a separate dependency?

405] How do you add Spring Boot Actuator to a project?

406] What are Actuator endpoints?

407] Name some commonly used Actuator endpoints.

408] What is the difference between /health and /info endpoints?

409] What does the /metrics endpoint provide?

410] What is the /env endpoint used for?

411] How do you access all endpoints of Actuator in Spring Boot 2.x?

412] What is the difference between “exposed” and “enabled” endpoints?

413] How do you enable or disable specific Actuator endpoints?

414] How do you change the default base path for Actuator endpoints?

415] What is the purpose of the /health endpoint?

416] How can you customize the /health endpoint?

417] What are Health Indicators in Spring Boot Actuator?

418] How do you create a custom Health Indicator?

419] How does Spring Boot Actuator collect metrics?

420] Name some default metrics provided by Spring Boot Actuator.

421] How can you create a custom metric using Actuator?

422] Are Actuator endpoints secure by default?

423] How can you secure Actuator endpoints in Spring Boot?

424] How do you configure role-based access to specific endpoints?

425] What is the difference between sensitive and non-sensitive endpoints?

426] How do you expose Actuator endpoints over HTTPS?

427] How can Actuator be used with monitoring tools like Prometheus, Grafana, or Micrometer?

428] What is Micrometer, and how does it relate to Spring Boot Actuator?

429] How can you collect JVM metrics using Actuator?

430] How do you monitor datasource health using Actuator?

431] How do you check cache health with Spring Boot Actuator?

432] How do you create a custom endpoint in Spring Boot Actuator?

433] What is the difference between @Endpoint and @RestController?

434] How do you define read, write, and delete operations in a custom Actuator endpoint?

435] How can you add custom tags to metrics exposed via Actuator?

436] How can you integrate Actuator with distributed tracing systems like Zipkin or Sleuth?

437] How would you troubleshoot performance issues using Actuator?

438] How can Actuator help in debugging a production application?

439] Can Actuator endpoints be exposed in production? If yes, what precautions should be taken?

440] How do you test custom Actuator endpoints in a Spring Boot application?

Spring Security

441] What is Spring Security and why is it used?

442] What are the main features of Spring Security?

443] How does Spring Security integrate with Spring Boot applications?

444] What is the difference between authentication and authorization?

445] Explain the security filter chain in Spring Security.

446] What are the different types of authentication supported by Spring Security?

447] How do you implement in-memory authentication?

448] How do you implement JDBC-based authentication?

449] How do you implement custom authentication logic?

450] How does Spring Security handle password encoding?

451] What is the difference between PasswordEncoder and BCryptPasswordEncoder?

452] How do you implement JWT-based authentication in Spring Boot REST APIs?

453] What is role-based access control (RBAC) in Spring Security?

454] How do you implement method-level security using @PreAuthorize and @Secured?

455] What is the difference between hasRole() and hasAuthority()?

456] How do you restrict access to certain endpoints based on roles?

457] What is OAuth2 and how is it used in Spring Security REST APIs?

458] What is the difference between OAuth2 and OpenID Connect?

459] How can you implement JWT + OAuth2 together in a Spring Boot application?

460] How do you secure REST endpoints in Spring Boot?

461] What is the difference between stateless and stateful authentication?

462] How do you implement stateless authentication using JWT?

463] How do you configure CORS in Spring Security for REST APIs?

464] How can you prevent CSRF attacks in Spring Boot REST APIs?

465] What are security filters in Spring Security?

466] What is the difference between UsernamePasswordAuthenticationFilter and BasicAuthenticationFilter?

467] How does the SecurityContextHolder work?

468] What are the common exceptions in Spring Security (e.g., AccessDeniedException, AuthenticationException)?

469] How can you implement multi-factor authentication (MFA) in Spring Boot?

470] How do you handle session management and concurrency control in Spring Security?

471] How do you unit test secured endpoints in Spring Boot?

472] How do you use @WithMockUser and @WithUserDetails for testing?

473] How do you test JWT-secured endpoints using MockMvc?

474] How do you test OAuth2-protected endpoints in Spring Boot?

475] How do you store passwords securely in Spring Boot applications?

476] How do you secure sensitive configuration properties?

477] How do you enable HTTPS in Spring Boot applications?

478] How do you log security events and handle auditing in Spring Security?

479] What are common security pitfalls to avoid in Spring Boot REST APIs?